SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	40% of CIA	**Total Marks:**	2 * 50 = 100 (1 Question1)
CO3 Statement:	*Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)*		
Student Name:	192472201	**Reg. No.:**	192472201
Section / Batch:	CSE - AI	**Date:**	17/08/26

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Implement the solution using **Python**.
- Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
- Use **embeddings and semantic search** where applicable.
- Use **FAISS or ChromaDB** as the vector database for RAG tasks.
- Demonstrate the complete pipeline:

Document → Chunking → Embedding → Retrieval → Generation

- Demonstrate the system with multiple queries.
- Handle irrelevant, incomplete, or unsupported queries appropriately.
- Display retrieved context/source wherever applicable.
- Explain the system architecture.
- Provide a working end-to-end demonstration.

Code of Conduct

I Y. Jayesh Ranjan – 192472201 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Jayesh Ranjan

SECTION A – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

26. RAG with Irrelevant Query Detection: Develop a RAG system that identifies questions not covered by the documents and responds appropriately instead of generating unsupported answers.

35. Multi-Tool RAG Agent: Develop an AI agent with at least three tools, such as document retrieval, calculator, and FAQ search. The agent should select the appropriate tool.

QUESTION 26 – RAG WITH IRRELEVANT QUERY DETECTION

AIM

To develop a Retrieval-Augmented Generation (RAG) system that retrieves relevant information from a document using semantic search and identifies irrelevant or unsupported queries instead of generating unsupported answers.

ALGORITHM

1. Start the program.
2. Load the document containing the knowledge base.
3. Divide the document into smaller chunks.
4. Load the Sentence Transformer embedding model.
5. Generate embeddings for all document chunks.
6. Store the embeddings in a FAISS vector database.
7. Accept a question from the user.
8. Generate an embedding for the user query.
9. Search FAISS to retrieve the most relevant document chunks.
10. Calculate the similarity distance between the query and retrieved chunks.
11. Compare the best similarity distance with a predefined relevance threshold.
12. If the query is relevant, generate an answer using the retrieved document content.
13. If the query is irrelevant, display that the information is not available in the provided document.
14. Display the retrieved context and source.
15. Repeat the process for multiple queries until the user enters exit.
16. Stop the program.

CODE

```
# ==============================================================
        print("Retrieved Text:", chunk)


    relevant = check_relevance(results)


    print("\n--- RELEVANCE CHECK ---")
```

```
    if relevant:

        print("STATUS: RELEVANT QUERY")

        answer = generate_answer(
            query,
            results
        )

        print("\n--- FINAL ANSWER ---")
        print(answer)

        print("\nSource: Course Notes")

    else:

        print("STATUS: IRRELEVANT QUERY")

        print("\n--- FINAL ANSWER ---")

        print(
            "Information not available in the "
            "provided document."
        )

        print(
            "The system did not generate an "
            "unsupported answer."
        )


# ============================================================
# 11. MAIN PROGRAM
# ============================================================
```

```
def main():

    print("\n================================================")
    print("RAG PIPELINE")
    print("================================================")

    print(
        "Document -> Chunking -> Embedding -> "
        "FAISS -> Retrieval -> Relevance Detection -> Answer"
    )

    print("\nType 'exit' to close the program.")

    while True:

        query = input(
            "\nEnter your question: "
        )

        if query.lower() == "exit":

            print("\nProgram ended.")
            break

        if query.strip() == "":

            print("Please enter a question.")
            continue

        ask_question(query)


# ==================================================================
# START
```

```
# ============================================================

if __name__ == "__main__":

    main()
```

INPUT

The user enters questions related to the document, for example:

What is paging?

What is FIFO page replacement?

What is virtual memory?

An irrelevant query can also be entered:

What is the capital of France?

The user can enter multiple queries. Enter exit to terminate the program.

OUTPUT

Result:

The program is executed sucesfully

QUESTION 35 – MULTI-TOOL RAG AGENT

AIM

To develop a multi-tool RAG-based AI agent that automatically selects and uses the appropriate tool for a user query, using document retrieval, calculator, and FAQ search tools.

ALGORITHM

1. Start the program.
2. Load the document knowledge base.
3. Create chunks from the document.
4. Generate embeddings for the document chunks.
5. Store the embeddings in a FAISS vector database.
6. Create three tools:
    - Document Retrieval Tool
    - Calculator Tool
    - FAQ Search Tool
7. Accept a query from the user.
8. Analyze the query to determine the appropriate tool.
9. If the query is a mathematical expression, select the Calculator Tool.
10. If the query is related to attendance, examination, library, course, or assignment information, select the FAQ Search Tool.
11. Otherwise, select the Document Retrieval Tool.
12. Retrieve relevant document information using FAISS when the document tool is selected.
13. Generate the result from the selected tool.
14. Display the selected tool and retrieved information.
15. Generate the final grounded response.
16. Handle unsupported document queries without generating false information.
17. Repeat for multiple queries.
18. Stop when the user enters exit.

CODE

```
# ==========================================================
            round(
                item["distance"],
                4
            )
        )
```

```
        print(
            "Text:",
            item["text"]
        )


    print("\n--- FINAL ANSWER ---")

    print(
        result["data"][0]["text"]
    )


# ============================================================
# 9. MAIN PROGRAM
# ============================================================

def main():

    print("\n=============================================")
    print("MULTI-TOOL RAG AGENT")
    print("=============================================")

    print("\nAvailable Tools:")

    print("1. Document Retrieval Tool")
    print("2. Calculator Tool")
    print("3. FAQ Search Tool")

    print("\nAgent automatically selects the tool.")

    print("\nType 'exit' to close the program.")

    while True:
```

```python
        query = input(
            "\nEnter your question: "
        )

        if query.lower() == "exit":

            print("\nProgram ended.")
            break

        if query.strip() == "":

            print(
                "Please enter a question."
            )

            continue

        run_agent(query)


# ============================================================
# START PROGRAM
# ============================================================

if __name__ == "__main__":

    main()
```

INPUT

The following queries can be used to demonstrate the three tools:

Document Retrieval Tool:

What is virtual memory?

Calculator Tool:

25 * 4

FAQ Search Tool:

What are the attendance rules?

Unsupported query:

What is the weather today?

Enter exit to terminate the program.

OUTPUT

Result :

The program is executed successfully.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Pipeline Functionality	30%	Correct RAG pipeline — embeddings, retrieval and generation all function coherently	Pipeline mostly correct with minor gaps	Pipeline only partially functional	Pipeline non-functional or conceptually flawed
Answer Relevance & Groundedness	25%	Retrieves relevant context and generates accurate, grounded answers	Mostly relevant/accurate answers	Inconsistent relevance or accuracy	Answers largely irrelevant or hallucinated
Query Robustness	20%	Robust handling of varied queries and documents	Handles most queries well	Handles only simple queries	Fails on basic queries
End-to-End Demo	15%	Smooth, well-demonstrated end-to-end system	Demo mostly smooth, minor hiccups	Demo has noticeable issues	Demo fails or is not ready
Architecture Explanation	10%	Clear architecture diagram and explanation of design decisions	Adequate explanation of design	Minimal explanation of design	No explanation of design provided

Marks Evaluation:

Question No.	Pipeline Functionality(30%)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	